A STUDENT HANDBOOK

University SQL Joins Handbook

8 join types, one real university database, every result explained row by row.

10	221	8
TABLES	ENROLLMENTS	JOIN PATTERNS

By the end of this handbook you will be able to look at a question like "which rooms are never used?" or "who reports to whom?", pick the right join without guessing, write it in MySQL, and predict what the result will look like before you run it. Every query here runs against the same small university database, and every result table shows real rows from it, so you can check yourself at each step.

Files used: 01_schema.sql, 02_data.sql, 03_joins.sql. MySQL 8.0 or newer.

PART 1: THE DATABASE

About the University Schema

One small university, ten tables, and a handful of deliberate quirks that make joins worth learning.

The database models a university the way a registrar's office would see it. There are **departments**, each with a building and a budget. **Professors** belong to a department, and every department has one Department Head that the other professors report to. **Students** usually belong to a department too, and each one may have an advisor from the professors table.

Teaching happens through **courses**. A course is a catalogue entry such as CS101. When it is actually taught, it becomes a **course section**: one offering in one semester, taught by one professor in one **room**. Students register for sections, and each registration is a row in **enrollments**, which carries the grade once the section is finished. Money flows through **scholarships** and **student_scholarships**, and academic progress is tracked over time in **semester_gpa**, one row per student per semester.

BUILT FOR TEACHING

Five features that are there on purpose

Real databases have gaps and odd cases. This one has five carefully placed ones, and each one is the reason a particular join type exists.

FEATURE	WHAT IT MEANS	JOIN IT DEMONSTRATES
`students.department_id` can be NULL	Students 73 to 77 are undeclared. They have no department and, on purpose, no enrollments at all.	LEFT JOIN, FULL OUTER
Room 15 is never used	The Library seminar room LIB-101 exists in `rooms` but no `course_sections` row points to it.	RIGHT JOIN
`professors.manager_id` points at professors	A self-referencing foreign key. The 8 Department Heads have `manager_id = NULL`; everyone else points to their head.	SELF JOIN
`enrollments.grade` can be NULL	Sections 32 to 35 are Fall 2024 and still running, so 32 enrollments have no grade yet.	INNER JOIN filter, COALESCE
`scholarships.department_id` can be NULL	NULL means the scholarship is university-wide rather than tied to one department.	CROSS JOIN, practice
`semester_gpa` is a time series	Up to 4 GPA rows per student. "Latest GPA" needs ROW_NUMBER before it can be joined.	NON-EQUI JOIN

WHY THIS MATTERS

Every NULL and every unused row in this database is a question waiting to be asked: "who has not enrolled?", "which room is empty?", "who has no boss?". Joins are how SQL answers those questions. Keep this table in mind; the rest of the handbook refers back to it.

PART 1: THE DATABASE

Tables and Their Features

Six lookup tables. These describe the people, places and things that the activity tables refer to.

departments `8 rows`

KEY COLUMNS	`department_id` PK, `dept_name`, `building`, `budget`
REFERS TO	Nothing. This is the root of the schema.
WHY IT MATTERS	Almost everything hangs off a department. Professors, students, courses and scholarships all point here.

rooms `15 rows`

KEY COLUMNS	`room_id` PK, `building`, `room_number`, `capacity`, `room_type`
REFERS TO	Nothing.
WHY IT MATTERS	Room 15 (LIB-101) is never used by any section. It is the star of the RIGHT JOIN example.

professors `20 rows`

KEY COLUMNS	`professor_id` PK, `first_name`, `last_name`, `title`, `salary`, `manager_id`
REFERS TO	`departments` and, through `manager_id`, itself.
WHY IT MATTERS	The self-referencing `manager_id` lets you build an org chart with a SELF JOIN. Heads have NULL there.

students `80 rows`

KEY COLUMNS	`student_id` PK, `first_name`, `last_name`, `department_id` (nullable), `status`, `advisor_id`
REFERS TO	`departments` (optional) and `professors` as advisor (optional).
WHY IT MATTERS	Five undeclared students have NULL department and no enrollments. They show what LEFT JOIN keeps and INNER JOIN drops.

courses `25 rows`

KEY COLUMNS	`course_id` PK, `course_code` (unique), `course_name`, `credits`, `level`
REFERS TO	`departments`.
WHY IT MATTERS	The catalogue. A course is not "taught" until a section exists for it, which is why transcripts go through `course_sections`.

scholarships `10 rows`

KEY COLUMNS	`scholarship_id` PK, `scholarship_name`, `amount`, `department_id` (nullable), `criteria`
REFERS TO	`departments` (optional; NULL means university-wide).
WHY IT MATTERS	Ten rows, which makes it the perfect partner for the CROSS JOIN: 80 students × 10 scholarships = 800 pairs.

HOW TO READ THESE CARDS

"PK" is the primary key, the column that identifies one row. "Refers to" lists the tables this one points at with a foreign key. When you join, you almost always match a foreign key on one side to a primary key on the other.

PART 1: THE DATABASE

Activity Tables

Four tables that record things happening: a section being taught, a student enrolling, money being awarded, a semester ending.

course_sections — 35 rows

KEY COLUMNS	section_id PK, course_id, professor_id, room_id, semester, year
REFERS TO	courses, professors, rooms.
WHY IT MATTERS	The bridge between the catalogue and the classroom. Sections 32 to 35 are Fall 2024 and still in progress.

enrollments — 221 rows

KEY COLUMNS	enrollment_id PK, student_id, section_id, grade (nullable), grade_points
REFERS TO	students, course_sections.
WHY IT MATTERS	The many-to-many link between students and sections. Most join examples start here. 32 rows have NULL grade.

student_scholarships — 40 rows

KEY COLUMNS	id PK, student_id, scholarship_id, academic_year, amount_awarded
REFERS TO	students, scholarships.
WHY IT MATTERS	Another many-to-many link. Compare it with the CROSS JOIN: 800 possible pairs, 40 actual awards.

semester_gpa — 121 rows

KEY COLUMNS	id PK, student_id, semester, year, semester_gpa, cumulative_gpa
REFERS TO	students.
WHY IT MATTERS	One row per student per semester. To join "the latest GPA" you first have to pick one row per student. See the NON-EQUI JOIN.

Lookup tables versus activity tables

A useful habit: before you write a join, decide which table is the **activity** you are reporting on and which tables are just **lookups** that add names and labels. In a transcript, `enrollments` is the activity; `students`, `courses` and `professors` are lookups. The activity table usually goes in the FROM clause, and one JOIN per lookup follows.

ROW COUNTS AT A GLANCE

DEPT-MENTS	ROOMS	PROF-ESSORS	STUDENTS	COURSES	SCHOLAR-SHIPS	COURSE_ SECTIONS	ENROLL-MENTS	STUDENT_ SCHOLAR-SHIPS	SEMESTER_ GPA
8	15	20	80	25	10	35	221	40	121

Knowing the counts lets you sanity-check a result. A join between students and enrollments should never return more than 221 matched rows, and a LEFT JOIN from students can only add 5 unmatched ones.

PART 1: THE DATABASE

What Refers to What

Every foreign key in `01_schema.sql`. These are the columns you will put in ON clauses.

CHILD COLUMN	PARENT COLUMN	NULLABLE?	WHY THE LINK EXISTS
professors.department_id	departments.department_id	No	Every professor belongs to exactly one department.
professors.manager_id	professors.professor_id	*Yes*	Self-reference. Points to the Department Head; NULL for the 8 heads.
students.department_id	departments.department_id	*Yes*	NULL means undeclared major (students 73 to 77).
students.advisor_id	professors.professor_id	*Yes*	Optional advisor. Undeclared students have none.
courses.department_id	departments.department_id	No	Each catalogue course is owned by one department.
scholarships.department_id	departments.department_id	*Yes*	NULL means university-wide, open to any department.
course_sections.course_id	courses.course_id	No	A section is always an offering of a real course.
course_sections.professor_id	professors.professor_id	No	Someone has to teach it.
course_sections.room_id	rooms.room_id	No	Every section meets in a room. Room 15 is simply never chosen.
enrollments.student_id	students.student_id	No	Who enrolled.
enrollments.section_id	course_sections.section_id	No	In which section. Together with student_id this pair is unique.
student_scholarships.student_id	students.student_id	No	Who received the award.
student_scholarships.scholarship_id	scholarships.scholarship_id	No	Which scholarship was awarded.
semester_gpa.student_id	students.student_id	No	Whose GPA history. Unique per student, semester and year.

READING A FOREIGN KEY

The child column holds a value that must exist in the parent column. So `enrollments.student_id = 13` promises that a students row with `student_id = 13` exists. That promise is exactly what makes `ON e.student_id = s.student_id` a safe join.

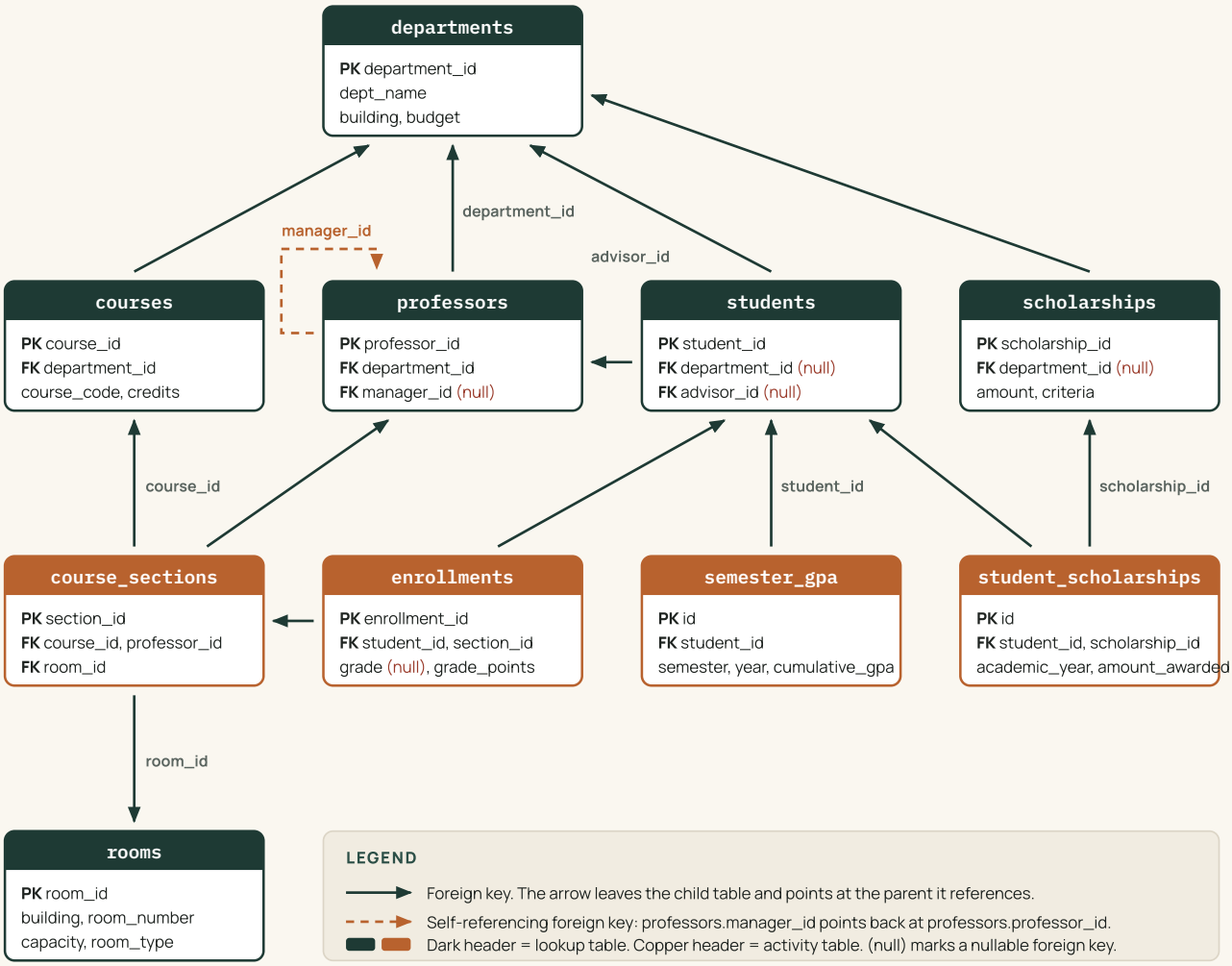
NULLABLE KEYS CHANGE THE JOIN YOU NEED

The four nullable foreign keys above are the ones where an INNER JOIN will lose rows. If the question is "all students" or "all professors", reach for LEFT JOIN whenever the link column can be NULL.

PART 1: THE DATABASE

Entity Relationship Diagram

All ten tables and the fourteen foreign keys that connect them. Arrows point from the child column to the parent table it references.



How to read it: start at the table you are reporting on and follow the arrows to the tables that give you names and labels. A transcript starts at `enrollments`, follows `student_id` to `students`, then `department_id` to `departments`, and follows `section_id` to `course_sections` and on to `courses`, `professors` and `rooms`. Each arrow you follow becomes one JOIN in your query.

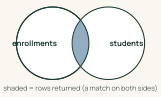
PART 2: THE EIGHT JOINS

1 INNER JOIN BASIC

The problem

The registrar wants a clean list of every completed enrollment: who took what, in which semester, and what grade they received. Enrollments that are still in progress should not appear, and neither should students who never enrolled.

enrollments	students	course_sections	courses
221 rows. The activity we report on. Holds student_id and section_id.	80 rows. Adds the name.	35 rows. Adds semester and year.	25 rows. Adds code and course name.



Only the overlap survives. An enrollment with a matching student, and a student with at least one enrollment. The five students with no enrollments sit in the white part of the right circle, so they are not returned. The same rule applies again for sections and for courses.

The SQL

```sql
SELECT
    CONCAT(s.first_name, ' ', s.last_name)  AS student_name,
    c.course_code,
    c.course_name,
    cs.semester,
    cs.year,
    e.grade
FROM enrollments       e
INNER JOIN students        s  ON e.student_id  = s.student_id
INNER JOIN course_sections cs ON e.section_id  = cs.section_id
INNER JOIN courses         c  ON cs.course_id  = c.course_id
WHERE e.grade IS NOT NULL
ORDER BY s.last_name, cs.year, cs.semester;
```

Four tables, three joins, one filter. Each JOIN line names the table to add and the pair of columns that must be equal.

1. INNER JOIN, CONTINUED

What comes back

RESULT (FIRST 7 OF 189 ROWS)

STUDENT_NAME	COURSE_CODE	COURSE_NAME	SEMESTER	YEAR	GRADE
Omar Abdullah	MATH101	Calculus I	Fall	2023	A
Omar Abdullah	MATH201	Calculus II	Spring	2024	B+
Laila Ahmed	PHYS101	General Physics I	Fall	2023	A
Laila Ahmed	PHYS201	General Physics II	Fall	2023	A-
Monica Allen	BIO101	General Biology	Fall	2023	B+
Monica Allen	BIO201	Cell Biology	Fall	2023	B
Monica Allen	BIO301	Genetics & Heredity	Spring	2024	B-

Notice what is missing. Omar Abdullah has three enrollments, but only two appear: his Fall 2024 course has no grade yet and the WHERE clause removed it. There is no row anywhere for Chris Morgan, Taylor Swift, Casey Jordan, Sam Patel or Alex Kim. Not one grade cell is NULL.

How it works

1. The engine starts with `enrollments`, the table in the FROM clause: 221 rows, each with a `student_id` and a `section_id` but no names.
2. For each enrollment it looks for a `students` row with the same `student_id`. INNER JOIN keeps the pair only when that match exists. If there were an enrollment with no student, it would be dropped here; and students with no enrollment never get pulled in at all.
3. The same matching happens again for `course_sections` (through `section_id`) and then `courses` (through `course_id`). Each join adds columns to the row being built.
4. `WHERE e.grade IS NOT NULL` then throws away the 32 enrollments from the Fall 2024 sections that are still running. 221 minus 32 leaves 189 rows.
5. `ORDER BY` sorts by last name, then year, then semester, which is why the list starts with Abdullah, Ahmed and Allen.

LEARNING POINT

INNER JOIN answers "show me the things that exist on both sides". If a row has no partner, it disappears without a message. That is exactly right for a grade list, and exactly wrong for a question that begins with "all students" or "every room".

COMMON MISTAKE: SILENT DROPS

INNER JOIN never tells you what it removed. There are 80 students, but only 75 can ever appear in this query. If a report comes back shorter than you expected, check whether a key on one side is NULL or has no partner before you blame the data.

PART 2: THE EIGHT JOINS

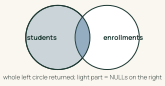
2 LEFT JOIN BASIC

The problem

An academic advisor asks: "Give me every student, and next to each one their enrollments. Students who have not registered for anything must still be on the list, because those are the ones I need to call."

students `left table`
80 rows. Every one of them must appear, even with nothing on the other side.

enrollments `right table`
221 rows. Attached where a match exists; NULL where it does not.



The whole left circle comes back. The darker overlap is the 221 student-enrollment pairs. The lighter crescent is the five students with no enrollment; they are returned with every enrollments column set to NULL. Nothing from the white part of the right circle can appear, because every enrollment has a student.

The SQL

```sql
SELECT
    s.student_id,
    CONCAT(s.first_name, ' ', s.last_name)  AS student_name,
    s.status,
    e.enrollment_id,
    e.grade
FROM students    s
LEFT JOIN enrollments e ON s.student_id = e.student_id
ORDER BY e.enrollment_id IS NULL DESC, s.last_name;
```

The ORDER BY trick: `e.enrollment_id IS NULL` evaluates to 1 for the unmatched rows and 0 for the rest, so `DESC` floats the students with no enrollment to the top.

2. LEFT JOIN, CONTINUED

What comes back

RESULT (FIRST 8 OF 226 ROWS)

STUDENT_ID	STUDENT_NAME	STATUS	ENROLLMENT_ID	GRADE
75	Casey Jordan	Active	*NULL*	*NULL*
77	Alex Kim	Active	*NULL*	*NULL*
73	Chris Morgan	Active	*NULL*	*NULL*
76	Sam Patel	Active	*NULL*	*NULL*
74	Taylor Swift	Active	*NULL*	*NULL*
13	Omar Abdullah	Active	41	A
13	Omar Abdullah	Active	42	B+
13	Omar Abdullah	Active	43	*NULL*

Two different kinds of NULL are on this page. Rows 1 to 5 have NULL in `enrollment_id`: no match was found, so the join filled the gap. Row 8 has a real enrollment (43) but a NULL grade: the match exists, the section is simply still in progress. Always test for "no match" on a key column, never on a data column.

How it works

1. Start from `students`, the table on the left of the JOIN keyword: 80 rows. LEFT JOIN guarantees every one of them appears at least once.
2. For each student, find the `enrollments` rows with the same `student_id`. A student with three enrollments produces three output rows, one per match.
3. No match? The engine still emits one row for the student and fills every column from `enrollments` with NULL. That is what happens for students 73 to 77.
4. 221 matched rows plus 5 unmatched students gives 226 rows. Compare that with 221 for the same tables under INNER JOIN.
5. ORDER BY puts the NULL rows first, then sorts by last name.

LEARNING POINT

LEFT JOIN means "everything from the left, plus whatever matches on the right". To find the rows with no partner, add `WHERE right_table.key IS NULL`, using a column that can never be NULL when a real match exists.

COMMON MISTAKE: WHERE TURNS LEFT INTO INNER

Write `WHERE e.grade = 'A'` after this LEFT JOIN and the five NULL rows fail the test and vanish. The query now behaves like an INNER JOIN and nobody warned you. Conditions that belong to the right table go in the ON clause, or you check for `IS NULL` explicitly.

2. LEFT JOIN, FOLLOW-UP

Counting with a LEFT JOIN

The advisor's real question was "how many courses is each student taking?" Same join, plus GROUP BY.

```sql
SELECT
    s.student_id,
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    COUNT(e.enrollment_id)                 AS courses_enrolled
FROM students    s
LEFT JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.first_name, s.last_name
ORDER BY courses_enrolled DESC;
```

RESULT (5 HIGHEST AND 5 LOWEST OF 80 ROWS)

STUDENT_ID	STUDENT_NAME	COURSES_ENROLLED
1	Alex Turner	4
2	Mei Lin	4
4	Fatima Hassan	4
7	Nathan Brooks	4
34	Christopher White	4
73	Chris Morgan	0
74	Taylor Swift	0
75	Casey Jordan	0
76	Sam Patel	0
77	Alex Kim	0

One row per student this time, 80 in total. The five undeclared students are still here, now with a clean 0 instead of a NULL. That 0 is the whole reason for using LEFT JOIN: with INNER JOIN they would not be in the list at all.

How it works

1. The LEFT JOIN builds the same 226 rows as before: 221 matched pairs and 5 students with NULLs.
2. GROUP BY folds those rows into one group per student. Omar Abdullah's three rows become one group.
3. `COUNT(e.enrollment_id)` counts only non-NULL values inside each group. The unmatched students have one row whose `enrollment_id` is NULL, so their count is 0.
4. ORDER BY courses_enrolled DESC puts the busiest students first and the five zeros last.

> **COMMON MISTAKE: COUNT(*) INSTEAD OF COUNT(COLUMN)**
>
> `COUNT(*)` counts rows, including the placeholder row with NULLs. It would report 1 course for Chris Morgan, which is wrong. After a LEFT JOIN, always count a column from the right table so the NULL rows count as zero.

PART 2: THE EIGHT JOINS

3 RIGHT JOIN BASIC

The problem

The facilities manager wants to know which rooms are actually being used and which ones sit empty. Every room must be on the list, including any room that no section has ever been scheduled in, because that is the room they can hand to another department.

course_sections `left table`
35 rows. Each one is taught in a room, so each carries a `room_id`.

rooms `right table`
15 rows. The side we want in full. Room 15 has no sections.



The mirror image of LEFT JOIN. All 15 rooms come back. Fourteen of them match at least one section and appear once per section (the darker overlap). Room 15 matches nothing and comes back once with NULL in every column that belongs to `course_sections`.

The SQL

```sql
SELECT
    r.room_id,
    r.building,
    r.room_number,
    r.room_type,
    r.capacity,
    cs.section_id,
    cs.semester,
    cs.year
FROM course_sections cs
RIGHT JOIN rooms r ON cs.room_id = r.room_id
ORDER BY cs.section_id IS NULL DESC, r.building, r.room_number;
```

Read it as: "take rooms in full, and attach any section whose room_id matches". The ORDER BY expression again floats the unmatched row to the top.

3. RIGHT JOIN, CONTINUED

What comes back

RESULT (FIRST 7 OF 36 ROWS)

ROOM_ID	BUILDING	ROOM_NUMBER	ROOM_TYPE	CAPACITY	SECTION_ID	SEMESTER	YEAR
15	Library	LIB-101	Seminar Room	20	*NULL*	*NULL*	*NULL*
11	Arts Building	ART-101	Seminar Room	35	19	Spring	2024
11	Arts Building	ART-101	Seminar Room	35	18	Fall	2023
12	Arts Building	ART-102	Seminar Room	20	21	Spring	2024
12	Arts Building	ART-102	Seminar Room	20	20	Fall	2023
8	Business Tower	BUS-101	Lecture Hall	150	26	Fall	2023
8	Business Tower	BUS-101	Lecture Hall	150	15	Spring	2024

Row 1 is the answer to the facilities manager's question: the Library seminar room has never hosted a section. Every other room repeats once per section held there, which is why BUS-101 shows up three times in the full result.

How it works

1. The right table is `rooms`, the one named after the RIGHT JOIN keyword. The engine promises every one of its 15 rows will appear.
2. For each room it looks for `course_sections` rows with the same `room_id`. A busy room such as BUS-101 matches three sections and produces three rows.
3. Room 15 matches nothing. It is still emitted once, with NULL in `section_id`, `semester` and `year`.
4. 35 sections plus 1 unused room gives 36 rows. The ORDER BY expression `cs.section_id IS NULL DESC` places the NULL row first, then sorts by building and room number.

LEARNING POINT

RIGHT JOIN is a LEFT JOIN read from the other end. `A RIGHT JOIN B` returns exactly the rows of `B LEFT JOIN A`. Most teams write only LEFT JOIN because reading left to right is easier, but you need to recognise RIGHT JOIN when you meet it in other people's code.

COMMON MISTAKE: WHICH SIDE IS "RIGHT"?

The right table is the one written after the RIGHT JOIN keyword, not the one you think of as more important. Swap the two table names by accident and you get all 35 sections with their rooms, and room 15 quietly disappears from the report.

PART 2: THE EIGHT JOINS

4 FULL OUTER JOIN INTERMEDIATE

The problem

An auditor wants everything on one screen: all students, all enrollments, and any record on either side that has no partner. Students who never enrolled must be visible, and so must any enrollment that points at a student who does not exist. Nothing may be dropped from either table.



Both circles, all of them. Matched pairs in the middle, students with no enrollment on the left with NULL enrollment columns, and enrollments with no student on the right with NULL student columns. In this database the foreign key makes the right crescent empty, but the query is written to catch it anyway.

> **MYSQL HAS NO FULL OUTER JOIN KEYWORD**
>
> Other databases let you write `FULL OUTER JOIN` directly. MySQL does not, so you build it from parts you already know: a LEFT JOIN for the left circle plus the middle, a RIGHT JOIN filtered down to the right crescent, and UNION ALL to stack them.

The SQL

```sql
SELECT
    s.student_id,
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    e.enrollment_id,
    e.section_id,
    e.grade
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id

UNION ALL

SELECT
    s.student_id,
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    e.enrollment_id,
    e.section_id,
    e.grade
FROM students s
RIGHT JOIN enrollments e ON s.student_id = e.student_id
WHERE s.student_id IS NULL

ORDER BY student_id, enrollment_id;
```

4. FULL OUTER JOIN, CONTINUED

What comes back

RESULT (ROWS 218 TO 224 OF 226 ROWS)

STUDENT_ID	STUDENT_NAME	ENROLLMENT_ID	SECTION_ID	GRADE
72	Blake Ross	218	3	B-
73	Chris Morgan	*NULL*	*NULL*	*NULL*
74	Taylor Swift	*NULL*	*NULL*	*NULL*
75	Casey Jordan	*NULL*	*NULL*	*NULL*
76	Sam Patel	*NULL*	*NULL*	*NULL*
77	Alex Kim	*NULL*	*NULL*	*NULL*
78	Robert Green	219	2	W

The combined list is sorted by student_id, so the five unmatched students sit between Blake Ross (72) and Robert Green (78). There are no rows with a NULL student_id, because the second half of the UNION found no orphaned enrollments. Total: 221 matched rows plus 5 students plus 0 orphans equals 226.

How it works

1. The first SELECT is the LEFT JOIN from section 2: all 80 students with their 221 enrollments, 226 rows, NULLs for the five who have none.
2. The second SELECT is a RIGHT JOIN: all 221 enrollments with their students. Its WHERE clause, `s.student_id IS NULL`, keeps only enrollments whose student could not be found.
3. Because `enrollments.student_id` is a foreign key, no such orphan can exist here. The second half returns 0 rows. In a messy database without foreign keys, this is where the bad data would surface.
4. `UNION ALL` stacks the two result sets. Both must have the same number of columns in the same order. ALL matters: plain UNION would also remove duplicates, which costs time and could hide legitimate repeated rows.
5. The final ORDER BY applies to the combined 226 rows, sorting by `student_id` and then `enrollment_id`.

LEARNING POINT

The MySQL recipe for a full outer join: LEFT JOIN, then UNION ALL, then the same RIGHT JOIN with a WHERE that keeps only the rows the LEFT JOIN could not have produced. Three pieces, each of which you already know.

COMMON MISTAKE: TESTING THE WRONG SIDE FOR NULL

The filter in the second half must check the LEFT table's key, `s.student_id IS NULL`. Test `e.student_id IS NULL` instead and you keep nothing, because every enrollment has a student_id. Drop the WHERE entirely and you duplicate all 221 matched rows.

PART 2: THE EIGHT JOINS

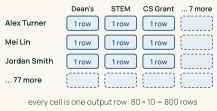
5 CROSS JOIN INTERMEDIATE

The problem

The financial aid office is building an eligibility worksheet. They want every student paired with every scholarship, so an officer can go down the list and tick which pairs meet the criteria. There is no key linking the two tables. We are not matching anything; we want every combination.

students
80 rows. One line in the worksheet per student per scholarship.

scholarships
10 rows. Three are university-wide, seven belong to a department.



A grid, not a Venn. There is nothing to overlap because nothing is being matched. Picture students down the side and scholarships across the top; the join fills in every cell. Ten cells per student, eighty students, eight hundred rows.

The SQL

```sql
SELECT
    s.student_id,
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    sc.scholarship_id,
    sc.scholarship_name,
    sc.amount
FROM students    s
CROSS JOIN scholarships sc
ORDER BY s.student_id, sc.scholarship_id;
```

No ON clause. That is not an omission; CROSS JOIN has no condition to check.

5. CROSS JOIN, CONTINUED

What comes back

RESULT (ROWS 1 TO 3 AND 10 TO 12 OF 800 ROWS)

STUDENT_ID	STUDENT_NAME	SCHOLARSHIP_ID	SCHOLARSHIP_NAME	AMOUNT
1	Alex Turner	1	Dean's Excellence Award	5000.00
1	Alex Turner	2	STEM Merit Scholarship	3500.00
1	Alex Turner	3	CS Innovation Grant	4000.00
. . . rows 4 to 9: Alex Turner with scholarships 4 to 9 . . .				
1	Alex Turner	10	Financial Need Assistance Fund	2000.00
2	Mei Lin	1	Dean's Excellence Award	5000.00
2	Mei Lin	2	STEM Merit Scholarship	3500.00

Alex Turner occupies rows 1 to 10, one per scholarship. Row 11 starts Mei Lin's block of ten. There are no NULLs anywhere, because no matching was attempted and so nothing could fail to match.

How it works

1. There is no ON clause, so there is nothing to compare. The engine takes the first student and pairs it with each of the 10 scholarships: 10 output rows.
2. It moves to the second student and does the same. After all 80 students, 80 × 10 = 800 rows exist.
3. No rows are filtered and no NULLs are produced. Every student column comes from a real student and every scholarship column from a real scholarship.
4. ORDER BY groups the rows by student and then by scholarship, which gives the neat blocks of ten in the sample.

LEARNING POINT

CROSS JOIN builds all combinations. It is the raw material for "which could pair with which" questions: student and scholarship, room and time slot, professor and course. Generate the grid first, then filter it with a WHERE clause (for example, keep only pairs where the scholarship's department is NULL or equals the student's department).

COMMON MISTAKE: ROW EXPLOSION

Row counts multiply. 80 × 10 is harmless; 10,000 × 10,000 is a hundred million rows and a very slow afternoon. Also, an INNER JOIN whose ON clause is wrong or missing behaves exactly like a CROSS JOIN. If a result is far larger than expected, check the ON clause first.

PART 2: THE EIGHT JOINS

6 SELF JOIN INTERMEDIATE

The problem

HR wants an org chart: each professor next to the name and title of the Department Head they report to. The catch is that both people live in the same `professors` table. A professor's `manager_id` is simply another professor's `professor_id`.

`professors AS p`
The professor being listed. We read `p.manager_id` from this copy.

`professors AS m`
The same table again, playing the manager. We read the head's name from this copy.



The table points at itself. Jennifer Park's `manager_id` is 1, and professor 1 is Robert Chen, a Department Head. To place his name on her row, the query opens `professors` a second time under the alias `m` and matches `p.manager_id` to `m.professor_id`.

The SQL

```sql
SELECT
    p.professor_id,
    CONCAT(p.first_name, ' ', p.last_name)   AS professor_name,
    p.title,
    p.salary,
    CONCAT(m.first_name, ' ', m.last_name)   AS department_head,
    m.title                                  AS head_title
FROM professors p
LEFT JOIN professors m ON p.manager_id = m.professor_id
ORDER BY p.department_id, p.professor_id;
```

Aliases are not optional here. Without `p` and `m`, MySQL cannot tell which copy of the table each column belongs to.

6. SELF JOIN, CONTINUED

What comes back

RESULT (FIRST 6 OF 20 ROWS)

PROFESSOR_ID	PROFESSOR_NAME	TITLE	SALARY	DEPARTMENT_HEAD	HEAD_TITLE
1	Robert Chen	Department Head	135000.00	*NULL*	*NULL*
2	Jennifer Park	Associate Professor	98000.00	Robert Chen	Department Head
3	Marcus Johnson	Assistant Professor	78000.00	Robert Chen	Department Head
4	Sarah Williams	Department Head	128000.00	*NULL*	*NULL*
5	Thomas Brown	Professor	110000.00	Sarah Williams	Department Head
6	Diana Lopez	Lecturer	65000.00	Sarah Williams	Department Head

Rows 1 and 4 are Department Heads. Their `manager_id` is NULL, so the second copy of the table found no partner and the head columns are NULL. The two professors under each head show the head's name pulled from the `m` copy. The full result has 20 rows, one per professor, including all 8 heads.

How it works

1. The engine opens `professors` twice and gives each copy its own name: `p` for the professor being listed and `m` for the manager.
2. For each row in `p`, it looks for a row in `m` where `m.professor_id` equals `p.manager_id`.
3. Jennifer Park has `manager_id = 1`, so she matches Robert Chen. His first and last name are concatenated into the `department_head` column on her row.
4. Robert Chen himself has `manager_id = NULL`. NULL never equals anything, so there is no match. Because the query uses LEFT JOIN, he stays in the result with NULL in the head columns.
5. 20 rows out, one per professor, sorted by department and then id, so each head is followed by the people who report to them.

LEARNING POINT

A self join is an ordinary join where both sides happen to be the same table. Aliases do the real work. Read the ON clause slowly, `p.manager_id = m.professor_id`, and it tells you which copy plays which role: `p` holds the pointer, `m` is the person pointed at.

COMMON MISTAKE: INNER JOIN HIDES THE BOSSES

With INNER JOIN instead of LEFT JOIN, the 8 Department Heads vanish because their NULL `manager_id` matches nothing. If an org-chart query returns fewer people than the table holds, you have lost the top of the hierarchy.

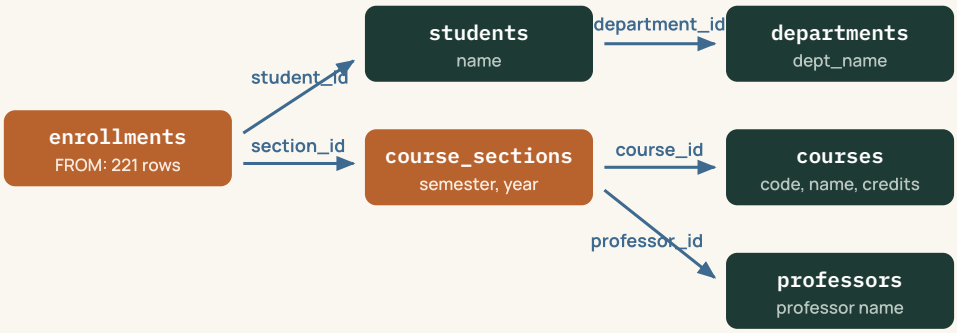
PART 2: THE EIGHT JOINS

7 MULTI-TABLE JOIN INTERMEDIATE

The problem

A student asks for a transcript: every course they have taken with its department, credits, professor, semester, year and grade. Courses still running should say "In Progress" rather than showing an empty cell. Pieces of that answer live in six different tables.

The join path



Each arrow is one JOIN and is labelled with the column that links the two tables. Start at `enrollments` and every other table is at most two hops away.

The SQL

```sql
SELECT
    CONCAT(s.first_name, ' ', s.last_name)   AS student_name,
    d.dept_name                              AS department,
    c.course_code,
    c.course_name,
    c.credits,
    CONCAT(p.first_name, ' ', p.last_name)  AS professor,
    cs.semester,
    cs.year,
    COALESCE(e.grade, 'In Progress')         AS grade,
    COALESCE(e.grade_points, '-')            AS grade_points
FROM enrollments       e
JOIN students          s  ON e.student_id  = s.student_id
JOIN course_sections   cs ON e.section_id  = cs.section_id
JOIN courses           c  ON cs.course_id  = c.course_id
JOIN professors        p  ON cs.professor_id = p.professor_id
JOIN departments       d  ON s.department_id = d.department_id
ORDER BY s.last_name, cs.year, cs.semester, c.course_code;
```

7. MULTI-TABLE JOIN, CONTINUED

What comes back

RESULT (FIRST 6 OF 221 ROWS)

STUDENT_NAME	DEPARTMENT	COURSE_CODE	COURSE_NAME	CREDITS	PROFESSOR	SEMESTER	YEAR	GRADE
Omar Abdullah	Mathematics	MATH101	Calculus I	4	Thomas Brown	Fall	2023	A
Omar Abdullah	Mathematics	MATH401	Real Analysis	3	Thomas Brown	Fall	2024	In Progress
Omar Abdullah	Mathematics	MATH201	Calculus II	4	Thomas Brown	Spring	2024	B+
Laila Ahmed	Physics	PHYS101	General Physics I	4	James Morrison	Fall	2023	A
Laila Ahmed	Physics	PHYS201	General Physics II	4	Rachel Adams	Fall	2023	A-
Laila Ahmed	Physics	PHYS301	Electromagnetism	3	James Morrison	Fall	2024	In Progress

Ten columns from six tables, and no NULLs on view: COALESCE turned the missing Fall 2024 grades into "In Progress" and the missing grade points into "-". All 221 enrollments appear, one row each. The sort is by last name, then year, then semester, which is why Fall 2024 sits between Fall 2023 and Spring 2024 for Omar.

How it works

1. Start at `enrollments`, the table that ties everything together. Each of its 221 rows is one student in one section.
2. Join `students` through `student_id` to get the name, then `departments` through `s.department_id` to get the department name. Note the second join uses a column from students, not from enrollments.
3. Join `course_sections` through `section_id`, then `courses` through `cs.course_id` for the code, name and credits.
4. Join `professors` through `cs.professor_id` to name the instructor. Plain `JOIN` means INNER JOIN, so each step keeps only matched rows.
5. `COALESCE(e.grade, 'In Progress')` returns the first non-NULL value it is given, which replaces the 32 NULL grades with readable text.

LEARNING POINT

A big join is just several small joins in a row. Draw the chain first: for each table you need, find the column that links it to a table already in the query. Then write one JOIN per link. The order of the JOIN lines does not change the answer, as long as each ON clause refers only to tables that appear above it.

COMMON MISTAKE: ONE INNER JOIN TOO MANY

Every JOIN here is an INNER JOIN. If an enrolled student had a NULL `department_id`, the departments join would silently drop their entire transcript. It works here only because the five undeclared students have no enrollments. In real data, use LEFT JOIN for any link that is optional.

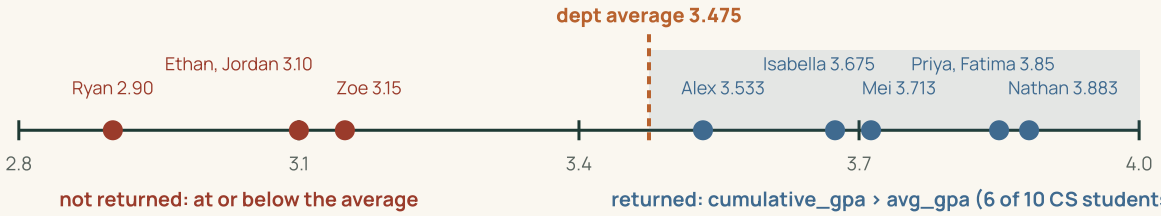
PART 2: THE EIGHT JOINS

8 NON-EQUI JOIN INTERMEDIATE

The problem

The dean wants to recognise students who are doing better than their peers: everyone whose latest cumulative GPA is above the average of their own department. Two things make this harder than it sounds. "Latest" GPA has to be picked out of a history table first, and the final test is a comparison, not an equality.

The threshold, drawn for Computer Science



Ten Computer Science students plotted on their latest cumulative GPA. The join keeps only the dots to the right of the department's average line. The same test runs separately inside each department.

The SQL

```sql
WITH latest_gpa AS (
    SELECT
        sg.student_id,
        sg.cumulative_gpa,
        ROW_NUMBER() OVER (
            PARTITION BY sg.student_id
            ORDER BY sg.year DESC, sg.semester DESC
        ) AS rn
    FROM semester_gpa sg
),
dept_avg AS (
    SELECT
        s.department_id,
        AVG(lg.cumulative_gpa) AS avg_gpa
    FROM latest_gpa    lg
    JOIN students      s  ON lg.student_id = s.student_id
    WHERE lg.rn = 1
      AND s.department_id IS NOT NULL
    GROUP BY s.department_id
)
SELECT
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    d.dept_name,
    lg.cumulative_gpa                      AS student_gpa,
    ROUND(da.avg_gpa, 3)                   AS dept_avg_gpa,
    ROUND(lg.cumulative_gpa - da.avg_gpa, 3) AS above_avg_by
FROM latest_gpa   lg
JOIN students     s  ON lg.student_id    = s.student_id
JOIN dept_avg     da ON s.department_id  = da.department_id
JOIN departments  d  ON s.department_id  = d.department_id
WHERE lg.rn = 1
  AND lg.cumulative_gpa > da.avg_gpa    -- <-- the non-equi condition
ORDER BY d.dept_name, above_avg_by DESC;
```

8. NON-EQUI JOIN, CONTINUED

What comes back

RESULT (FIRST 7 OF 22 ROWS)

STUDENT_NAME	DEPT_NAME	STUDENT_GPA	DEPT_AVG_GPA	ABOVE_AVG_BY
Stephanie Lewis	Biology	3.913	3.844	0.069
Melissa Johnson	Business Administration	3.875	3.404	0.471
Christopher White	Business Administration	3.800	3.404	0.396
Victoria Chen	Business Administration	3.650	3.404	0.246
Nathan Brooks	Computer Science	3.883	3.475	0.408
Fatima Hassan	Computer Science	3.850	3.475	0.375
Priya Nair	Computer Science	3.850	3.475	0.375

Every `student_gpa` is larger than the `dept_avg_gpa` beside it; that is the join condition made visible. The average differs per department (3.844 for Biology, 3.404 for Business), so a 3.65 makes the list in Business but would not in Biology. 22 students clear the bar across all 8 departments.

How it works

1. The CTE `latest_gpa` numbers each student's GPA rows from newest to oldest with `ROW_NUMBER()`. The row with `rn = 1` is the latest semester.
2. The CTE `dept_avg` joins those rows to `students`, keeps only `rn = 1` and students with a department, and averages the latest GPA per department. Eight rows, one per department.
3. The final SELECT joins each student's latest GPA to their own department's average. The three JOIN lines are ordinary equality joins on `student_id` and `department_id`.
4. The non-equi part is the last condition: `lg.cumulative_gpa > da.avg_gpa`. A comparison, not an equality, decides which of the joined rows survive. With INNER JOIN, a condition in WHERE and a condition in ON produce the same rows, so it is fine to write it here.
5. ORDER BY groups the result by department and, within each, puts the biggest margin first.

LEARNING POINT

A join condition does not have to be "=". Any true or false test works: `>`, `<`, `BETWEEN`, a date range, a threshold. The equality joins bring the right numbers onto the same row; the inequality then asks the real question.

COMMON MISTAKE: FORGETTING RN = 1

Drop `WHERE lg.rn = 1` from the final query and every semester's GPA is compared, so one student can appear several times, sometimes with a GPA from a year ago. The filter appears twice on purpose: once when averaging and once when selecting.

PART 3: QUICK REFERENCE

Join Cheat Sheet

All eight patterns on one page. Keep this one open while you practise.

JOIN TYPE	KEEPS UNMATCHED ROWS FROM	TYPICAL QUESTION	MYSQL NOTE
INNER JOIN	Neither side	Grades for every completed enrollment.	`JOIN` on its own means INNER JOIN.
LEFT JOIN	The left table	All students, even the ones with no enrollment.	Find the gaps with `right.key IS NULL`. Count with `COUNT(right.column)`.
RIGHT JOIN	The right table	All rooms, even the ones never scheduled.	Same as LEFT JOIN with the tables swapped. Most style guides prefer LEFT.
FULL OUTER JOIN	Both tables	All students and all enrollments, including orphans on either side.	No keyword. Use LEFT JOIN, UNION ALL, RIGHT JOIN with `WHERE left.key IS NULL`.
CROSS JOIN	Nothing is matched; every pair is returned	Every student paired with every scholarship.	No ON clause. Rows multiply: 80 × 10 = 800.
SELF JOIN	Depends on the join type you use	Each professor with their Department Head.	Same table twice with two aliases. Use LEFT JOIN to keep the top level.
MULTI-TABLE JOIN	Depends on each join in the chain	A transcript built from six tables.	One JOIN per link. Each ON clause may only use tables already listed above it.
NON-EQUI JOIN	Neither side (with INNER)	Students above their department's average GPA.	Condition uses `>`, `<` or `BETWEEN`. Often needs a CTE to prepare one row per key first.

Which join do I need?

- Does the question start with "all" or "every" about one table? Put that table on the left and use **LEFT JOIN**.
- Do you only care about rows that exist on both sides, such as completed grades or awarded scholarships? Use **INNER JOIN**.
- Do you need "all of A and all of B, including the gaps on both sides"? Use the **FULL OUTER** recipe with UNION ALL.
- Is there no key linking the tables and you want every combination? Use **CROSS JOIN**, then filter.
- Does the other row you need live in the same table (a manager, a previous semester, a sibling record)? Use a **SELF JOIN** with two aliases.
- Is the link a comparison rather than an equality (above average, within a date range)? That is a **NON-EQUI JOIN**; prepare one row per key with a CTE first.

> **THREE QUESTIONS BEFORE ANY JOIN**
>
> Which table is the activity I am reporting on? Which column links each lookup to it? Can that column be NULL? The answers tell you the FROM table, every ON clause, and whether to write INNER or LEFT.

PRACTICE

Practice questions

Ten requests from around the university, ordered from easy to intermediate. Every answer is shown as a result table, not as SQL, so you have to build the query yourself.

How to use these pages

1. **Try before you look.** Read the question, decide which tables you need and which join keeps the rows you care about, then write and run the query on the university database.
2. **Compare row counts first.** Each answer states its exact number of rows. Too few rows usually means an INNER JOIN or a WHERE clause quietly dropped something. Too many usually means a missing filter or a missing join condition.
3. **Then compare cells.** Highlighted cells are the ones that prove the point: a NULL in an unmatched row, a zero in a count, a row that only exists on one side. If your output has the same rows and the same highlighted values, you got it.

THE TEN QUESTIONS AT A GLANCE

#	TITLE	JOIN TO THINK ABOUT	ROWS
Q1	Courses in the Engineering Building	INNER JOIN, two tables	6
Q2	Who teaches in Business Tower	INNER JOIN, filter on the joined table	5
Q3	Students and their advisors	LEFT JOIN, keep the unmatched row	7
Q4	Economics students with no scholarship	LEFT JOIN plus IS NULL	5
Q5	How busy is each seminar room	LEFT JOIN plus COUNT, keep the zero	5
Q6	Data Science teaching load	RIGHT JOIN, three tables	4
Q7	Department Heads and their teams	SELF JOIN with a count	8
Q8	Grading patterns in Science Hall	Four tables plus AVG and GROUP BY	5
Q9	Physics students this Fall	LEFT JOIN, where the filter belongs	8
Q10	Departments and scholarships, both sides	FULL OUTER JOIN built with UNION	11

BEFORE YOU START

Keep the schema page open. Every question names the columns it expects in the output, so you always know what shape the answer should have. Column names in the answer tables are suggestions. Your aliases may differ, but the values must match.

A HABIT WORTH BUILDING

When a question says "include the ones that have none" or "even if", that is your signal to reach for an outer join. When it says "only the ones that have", an INNER JOIN or an IS NULL check is usually what you want.

Q1 Courses in the Engineering Building

EASY

The facilities office is printing a course board for the Engineering Building lobby. They want every course that belongs to a department housed in that building. Show `course_code`, `course_name`, `dept_name` and `building`, sorted by course code.

HINT

Two tables: `courses` and `departments`. The building lives in `departments`, so join first, then filter. Every course has a department, so an INNER JOIN loses nothing here.

EXPECTED ANSWER (6 ROWS)

COURSE_CODE	COURSE_NAME	DEPT_NAME	BUILDING
CS101	Introduction to Programming	Computer Science	Engineering Building
CS201	Data Structures & Algorithms	Computer Science	Engineering Building
CS301	Database Systems	Computer Science	Engineering Building
CS401	Operating Systems	Computer Science	Engineering Building
CS501	Machine Learning	Computer Science	Engineering Building
DS201	Python for Data Science	Data Science	Engineering Building

CHECK YOURSELF

Two departments share this building, so the last row is Data Science, not Computer Science. If you got 25 rows you forgot the building filter. If you got 200 rows the join has no ON condition and every course was paired with every department.

Q2 Who teaches in Business Tower

EASY

The dean's office needs a staff list for Business Tower, the building shared by Business Administration and Economics. List each professor's full name as `professor_name`, plus `title`, `dept_name` and `building`. Sort by department name, then by last name.

HINT

`professors` and `departments`, joined on `department_id`. Notice that `professors` has no building column at all. You only get access to it after the join.

EXPECTED ANSWER (5 ROWS)

PROFESSOR_NAME	TITLE	DEPT_NAME	BUILDING
Emily Rodriguez	Department Head	Business Administration	Business Tower
Nicole Turner	Associate Professor	Business Administration	Business Tower
Kevin Zhang	Professor	Business Administration	Business Tower
David Kim	Department Head	Economics	Business Tower
Priya Sharma	Assistant Professor	Economics	Business Tower

CHECK YOURSELF

If you got 20 rows, the building filter is missing. If you got only the three Business Administration rows, you filtered on department name instead of building and missed the two Economics professors who sit in the same tower.

Q3 Students and their advisors

EASY

An advisor is checking who looks after the students whose last name starts with M. She wants every one of those students, even the ones who have not been assigned an advisor yet. Show `student_id`, `student_name`, `advisor_name` and the advisor's `title`, sorted by student id.

HINT

`students` joined to `professors` through `advisor_id`. The phrase "even the ones who have not been assigned" tells you which join type to pick, and which table must be on the left.

EXPECTED ANSWER (7 ROWS)

STUDENT_ID	STUDENT_NAME	ADVISOR_NAME	TITLE
8	Zoe Mitchell	Marcus Johnson	Assistant Professor
11	Lucas Martini	Thomas Brown	Professor
16	Benjamin Moore	Thomas Brown	Professor
38	Joshua Martinez	Kevin Zhang	Professor
45	Emily Morris	Sophia Martinez	Assistant Professor
65	Alicia Morgan	Hannah White	Lecturer
73	Chris Morgan	*NULL*	*NULL*

CHECK YOURSELF

If you used an INNER JOIN you got 6 rows and Chris Morgan disappeared, because his `advisor_id` is NULL and NULL never matches anything. The LEFT JOIN keeps him and fills the advisor columns with NULL.

Q4 Economics students with no scholarship

EASY

The financial aid office wants to reach out to Economics students who have never received any scholarship, not even once. Show `student_id`, `student_name` and `status`, sorted by student id.

HINT

`students` and `student_scholarships`. You are looking for students with no matching award row. Think about which join produces those unmatched rows, and how you then keep only them.

EXPECTED ANSWER (5 ROWS)

STUDENT_ID	STUDENT_NAME	STATUS
57	Michelle Turner	Active
59	Lauren Baker	Active
61	Tiffany Robinson	Active
62	Kyle Harris	Suspended
63	Samantha Cox	Active

CHECK YOURSELF

An INNER JOIN can never find these students. It returns only the 4 award rows belonging to the 3 Economics students who did receive money (Andrew Carter, Jason Phillips and Ryan Nelson). The students you want exist only as unmatched rows in a LEFT JOIN, which is why the IS NULL test comes after it.

Q5 How busy is each seminar room

MEDIUM

The timetabling team wants to know how many course sections each seminar room has hosted across all semesters. Rooms that have never been used must still appear, showing zero. Show `room_number`, `building`, `capacity` and `sections_hosted`, busiest room first, then by room number.

HINT

`rooms` on the left, `course_sections` on the right, then GROUP BY room. Be careful about what you put inside COUNT. Counting every row and counting a column from the sections table give different answers for an unused room.

EXPECTED ANSWER (5 ROWS)

ROOM_NUMBER	BUILDING	CAPACITY	SECTIONS_HOSTED
BUS-102	Business Tower	25	3
SCI-102	Science Hall	30	3
ART-101	Arts Building	35	2
ART-102	Arts Building	20	2
LIB-101	Library	20	0

CHECK YOURSELF

With an INNER JOIN you get 4 rows and the Library room is gone. With a LEFT JOIN but `COUNT(*)`, the Library shows 1 instead of 0, because the unmatched row still counts as a row. Only counting a sections column gives the honest zero.

Q6 Data Science teaching load

MEDIUM

The head of Data Science wants a list of every professor in her department alongside each section they have taught, and she insists on seeing professors who have not taught anything yet. Show `professor_name`, `title`, `course_code`, `semester` and `year`. Start the query from `course_sections` and write it as a RIGHT JOIN so professors are the side that is fully kept. Sort by professor id, then year, then semester.

HINT

Three tables: `course_sections`, `courses` and `professors`. The RIGHT JOIN goes on the table whose rows must all survive. The department filter belongs to that table, so it is safe in the WHERE clause.

EXPECTED ANSWER (4 ROWS)

PROFESSOR_NAME	TITLE	COURSE_CODE	SEMESTER	YEAR
Aisha Patel	Department Head	DS201	Fall	2023
Aisha Patel	Department Head	DS201	Spring	2024
Carlos Rivera	Associate Professor	DS201	Spring	2024
Hannah White	Lecturer	*NULL*	*NULL*	*NULL*

CHECK YOURSELF

An INNER JOIN returns 3 rows and Hannah White is missing, because no section points at her. She is the only professor in the whole university who has never been assigned a section, so this is the one row a RIGHT JOIN adds.

Q7 Department Heads and their teams

MEDIUM

Human Resources is reviewing spans of control. For every Department Head, show how many professors report directly to them. Output `head_name`, `dept_name` and `direct_reports`, largest team first, then by the head's last name.

HINT

The reporting line is inside `professors` itself: `manager_id` points at another professor. Use the table twice with two different aliases, one for the head and one for the people under them, and bring in `departments` for the name.

EXPECTED ANSWER (8 ROWS)

HEAD_NAME	DEPT_NAME	DIRECT_REPORTS
Robert Chen	Computer Science	2
Aisha Patel	Data Science	2
Emily Rodriguez	Business Administration	2
Sarah Williams	Mathematics	2
Lisa Chen	Biology	1
David Kim	Economics	1
James Morrison	Physics	1
Michael Thompson	English Literature	1

CHECK YOURSELF

The counts must add up to 12, the number of professors who are not heads. If you see 12 rows with two names each, you listed the reports instead of counting them. If you see all 20 professors, you forgot to keep only Department Heads on the manager side.

VARIANT TO TRY

Flip the question around. Instead of counting, list the professors who report to Sarah Williams, the head of Mathematics, with their title and the head's name in a column called `reports_to`. Same two aliases, but now the WHERE clause filters on the head alias and the SELECT reads from both.

EXPECTED ANSWER FOR THE VARIANT (2 ROWS)

PROFESSOR_NAME	TITLE	REPORTS_TO
Thomas Brown	Professor	Sarah Williams
Diana Lopez	Lecturer	Sarah Williams

Both columns come from the same physical table. The aliases are what tell SQL which copy is the head and which copy is the report.

Q8 Grading patterns in Science Hall

INTERMEDIATE

The provost is comparing grading across the two departments in Science Hall, Mathematics and Physics. For each professor there, show `professor_name`, `dept_name`, `sections_taught`, `graded_enrollments` (enrollments with a grade) and `avg_grade_points` rounded to 2 decimals. Highest average first.

HINT

Four tables in a chain: `professors`, `departments`, `course_sections`, `enrollments`. One professor teaches many sections and each section has many enrollments, so think about DISTINCT when you count sections. AVG skips NULL grade points on its own.

EXPECTED ANSWER (5 ROWS)

PROFESSOR_NAME	DEPT_NAME	SECTIONS_TAUGHT	GRADED_ENROLLMENTS	AVG_GRADE_POINTS
Sarah Williams	Mathematics	1	8	3.46
Thomas Brown	Mathematics	3	23	3.37
Rachel Adams	Physics	2	10	3.37
Diana Lopez	Mathematics	1	5	3.26
James Morrison	Physics	2	8	3.20

CHECK YOURSELF

If Thomas Brown shows 23 sections instead of 3, you counted enrollment rows, not distinct sections. The join multiplies each section by its enrollments, and a plain COUNT counts the multiplied rows. Brown and Morrison both have a Fall 2024 section still in progress, which is why their graded count is lower than their total enrollment count.

SEE THE FAN-OUT

Here is Thomas Brown's row taken apart, one line per section. Three sections, 25 enrollment rows, 23 of them graded. After the join to `enrollments`, Brown appears 25 times, not 3, which is why a plain COUNT of his sections is wrong and a DISTINCT count is right.

THOMAS BROWN, SECTION BY SECTION (3 ROWS)

COURSE_CODE	SEMESTER	YEAR	ENROLLMENTS	GRADED
MATH101	Fall	2023	14	14
MATH201	Spring	2024	9	9
MATH401	Fall	2024	2	0

The Fall 2024 section is still running, so its two enrollments have no grade yet. AVG ignores them, and so does COUNT on the grade_points column.

Q9 Physics students this Fall

INTERMEDIATE

The Physics department wants a registration check for Fall 2024. For every Physics student, show how many Fall 2024 sections they are enrolled in. Students with nothing this term are the whole point of the report, so they must appear with a zero. Output `student_id`, `student_name`, `status` and `fall_2024_courses`, sorted by student id.

HINT

Three tables: `students`, `enrollments`, `course_sections`. Both joins must be LEFT JOINs. The trap is where you put the Fall 2024 condition. A WHERE clause runs after the join and throws away every row whose semester is NULL, which is exactly the rows for students with no enrollment this term. Attach the semester test to the join instead, and count a column from `course_sections`.

EXPECTED ANSWER (8 ROWS)

STUDENT_ID	STUDENT_NAME	STATUS	FALL_2024_COURSES
21	Christine Wu	Active	0
22	Patrick OBrien	Active	0
23	Laila Ahmed	Active	1
24	Marcus Stone	Graduated	0
25	Sophie Laurent	Active	0
26	Diego Ramirez	Active	0
27	Anna Kowalski	Active	1
28	James Foster	Active	0

CHECK YOURSELF

If your result has only 2 rows (Laila Ahmed and Anna Kowalski), the semester filter is in the WHERE clause. The LEFT JOIN did its job and kept all 8 students, but the WHERE clause then removed the 6 rows where `semester` was NULL. Same tables, same join type, wrong place for the condition.

WHY THIS MATTERS

This is the most common outer-join bug in real reports. The query runs without error and the numbers it shows are correct, so nobody notices that six students silently vanished. Whenever you filter on a column from the optional side of a LEFT JOIN, ask whether the filter belongs in ON rather than WHERE.

SAME JOIN, FILTER IN THE WRONG PLACE (2 ROWS)

STUDENT_ID	STUDENT_NAME	FALL_2024_COURSES
23	Laila Ahmed	1
27	Anna Kowalski	1

Six Physics students are missing from this version, and nothing in the output warns you.

Q10 Departments and scholarships, both sides

INTERMEDIATE

The scholarship committee wants one complete picture: every department next to the scholarship that belongs to it, plus departments that have no scholarship of their own, plus scholarships that are open to the whole university and belong to no department. Nothing from either table may be left out. Show `dept_name`, `scholarship_name` and `amount`. Put the department rows first in alphabetical order, then the university-wide rows.

HINT

`departments` and `scholarships`, joined on `department_id`. MySQL has no FULL OUTER JOIN keyword, so build it the way the handbook showed: a LEFT JOIN, then UNION ALL with a RIGHT JOIN that keeps only the rows the left side could not match.

EXPECTED ANSWER (11 ROWS)

DEPT_NAME	SCHOLARSHIP_NAME	AMOUNT
Biology	Biological Sciences Grant	3500.00
Business Administration	Business Leadership Scholarship	3000.00
Computer Science	CS Innovation Grant	4000.00
Data Science	*NULL*	*NULL*
Economics	Economics Policy Award	3000.00
English Literature	Humanities Excellence Award	2500.00
Mathematics	Mathematics Achievement Award	3000.00
Physics	Physics Research Fellowship	4500.00
NULL	Dean's Excellence Award	5000.00
NULL	Financial Need Assistance Fund	2000.00
NULL	STEM Merit Scholarship	3500.00

CHECK YOURSELF

Count the rows from each half. An INNER JOIN gives 7 matched pairs. A LEFT JOIN alone gives 8, adding Data Science with NULL scholarship. A RIGHT JOIN alone gives 10, adding the three university-wide scholarships with NULL department. Only the union of both outer sides reaches 11. If you got 18, you used UNION ALL without the IS NULL filter on the second half and the 7 matched pairs were listed twice.

READ THE NULLS

A NULL on the right means "this department has nothing". A NULL on the left means "this scholarship belongs to nobody in particular". Same symbol, opposite story. That is the whole reason a full outer join exists: to show both kinds of gaps in one table.

BEFORE YOU GO

What you can now do

- Look at a question and say which join it needs, and why, before touching the keyboard.
- Predict the row count of a join from the table sizes and the NULLs in the key columns.
- Tell a "no match" NULL from a "missing data" NULL, and test for the right one.
- Build a FULL OUTER JOIN in MySQL from LEFT JOIN, UNION ALL and RIGHT JOIN.
- Chain six tables into one transcript query without losing rows you did not mean to lose.

How to practise

Load `01_schema.sql`, then `02_data.sql`, into MySQL 8. Run each query in `03_joins.sql` and compare your output with the result tables in this handbook, row by row. Then work through the practice questions. Write your own query first, check the row count against your prediction, and only then look at the answer.

QUESTIONS, CORRECTIONS, MORE SQL

linkedin.com/in/sajibkhansk

Connect and share what you built with this dataset.